

Path 1: Pre Security (Legacy) > Module 3: How The Web Works > Room 2: HTTP in Detail

<https://tryhackme.com/room/httpindetail>

HTTP in Detail:

Learn about how you request content from a web server using the HTTP protocol.

>> Task 1: What is HTTP(S)?

- ♦ **What is HTTP? (HyperText Transfer Protocol):**

HTTP is what's used whenever you view a website, developed by Tim Berners-Lee and his team between 1989-1991. HTTP is the set of rules used for communicating with web servers for the transmitting of webpage data, whether that is HTML, Images, Videos, etc.

- ♦ **What is HTTPS? (HyperText Transfer Protocol Secure):**

HTTPS is the secure version of HTTP. HTTPS data is encrypted so it not only stops people from seeing the data you are receiving and sending, but it also gives you assurances that you're talking to the correct web server and not something impersonating it.

Q1) What does HTTP stand for?

Answer: HyperText Transfer Protocol

Q2) What does the S in HTTPS stand for?

Answer: Secure

Q3) On the mock webpage on the right there is an issue, once you've found it, click on it. What is the challenge flag?

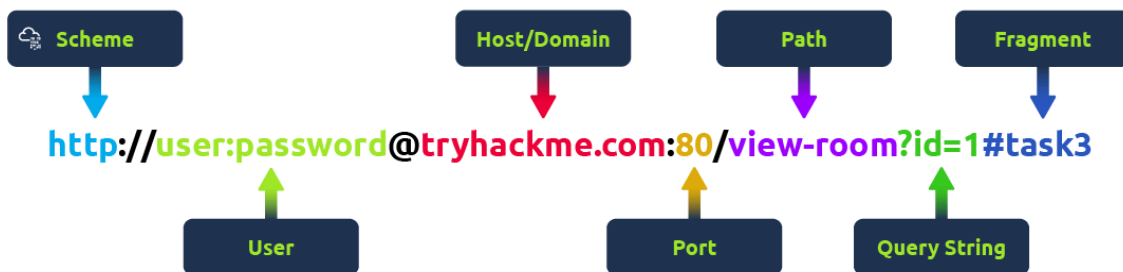
Answer: THM{INVALID_HTTP_CERT}

>> Task 2: Requests And Responses

When we access a website, your browser will need to make requests to a web server for assets such as HTML, Images, and download the responses. Before that, you need to tell the browser specifically how and where to access these resources, this is where URLs will help.

♦ What is a URL? (Uniform Resource Locator):

If you've used the internet, you've used a URL before. A URL is predominantly an instruction on how to access a resource on the internet. The below image shows what a URL looks like with all of its features (it does not use all features in every request).



Scheme: This instructs on what protocol to use for accessing the resource such as HTTP, HTTPS, FTP (File Transfer Protocol).

User: Some services require authentication to log in, you can put a username and password into the URL to log in.

Host: The domain name or IP address of the server you wish to access.

Port: The Port that you are going to connect to, usually 80 for HTTP and 443 for HTTPS, but this can be hosted on any port between 1 - 65535.

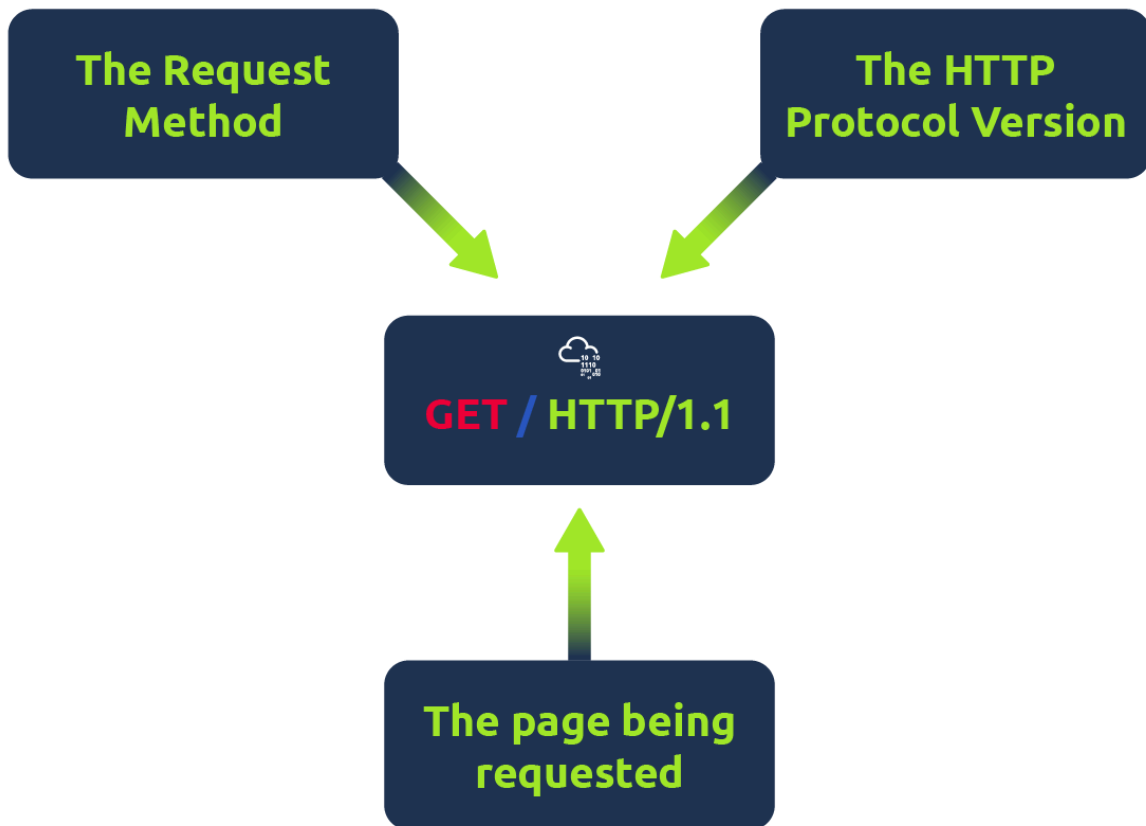
Path: The file name or location of the resource you are trying to access.

Query String: Extra bits of information that can be sent to the requested path. For example, `/blog?id=1` would tell the blog path that you wish to receive the blog article with the id of 1.

Fragment: This is a reference to a location on the actual page requested. This is commonly used for pages with long content and can have a certain part of the page directly linked to it, so it is viewable to the user as soon as they access the page.

- ◆ **Making a Request:**

It's possible to make a request to a web server with just one line GET / HTTP/1.1



But for a much richer web experience, you'll need to send other data as well. This other data is sent in what is called headers, where headers contain extra information to give to the web server you're communicating with, but we'll go more into this in the Header task.

- ◆ **Example Request:**

GET / HTTP/1.1

Host: tryhackme.com

User-Agent: Mozilla/5.0 Firefox/87.0

Referer: <https://tryhackme.com/>

To breakdown each line of this request:

Line 1: This request is sending the GET method (more on this in the HTTP Methods task), request the home page with / and telling the web server we are using HTTP protocol version 1.1.

Line 2: We tell the web server we want the website tryhackme.com

Line 3: We tell the web server we are using the Firefox version 87 Browser

Line 4: We are telling the web server that the web page that referred us to this one is <https://tryhackme.com>

Line 5: HTTP requests always end with a blank line to inform the web server that the request has finished.

♦ **Example Response:**

HTTP/1.1 200 OK

Server: nginx/1.15.8

Date: Fri, 09 Apr 2021 13:34:03 GMT

Content-Type: text/html

Content-Length: 98

```
<html>
<head>
  <title>TryHackMe</title>
</head>
<body>
  Welcome To TryHackMe.com
</body>
</html>
```

To breakdown each line of the response:

Line 1: HTTP 1.1 is the version of the HTTP protocol the server is using and then followed by the HTTP Status Code in this case "200 OK" which tells us the request has completed successfully.

Line 2: This tells us the web server software and version number.

Line 3: The current date, time and timezone of the web server.

Line 4: The Content-Type header tells the client what sort of information is going to be sent, such as HTML, images, videos, pdf, XML.

Line 5: Content-Length tells the client how long the response is, this way we can confirm no data is missing.

Line 6: HTTP response contains a blank line to confirm the end of the HTTP response.

Lines 7-14: The information that has been requested, in this instance the homepage.

Q1) What HTTP protocol is being used in the above example?

Answer: HTTP/1.1

Q2) What response header tells the browser how much data to expect?

Answer: Content-Length

>> Task 3: HTTP Methods

HTTP methods are a way for the client to show their intended action when making an HTTP request. There are a lot of HTTP methods but we'll cover the most common ones, although mostly you'll deal with the GET and POST method.

GET Request:

This is used for getting information from a web server.

POST Request:

This is used for submitting data to the web server and potentially creating new records

PUT Request:

This is used for submitting data to a web server to update information

DELETE Request:

This is used for deleting information/records from a web server.

Q1) What method would be used to create a new user account?

Answer: POST

Q2) What method would be used to update your email address?

Answer: PUT

Q3) What method would be used to remove a picture you've uploaded to your account?

Answer: DELETE

Q4) What method would be used to view a news article?

Answer: GET

>> Task 4: HTTP Status Codes

♦ HTTP Status Codes:

In the previous task, you learnt that when a HTTP server responds, the first line always contains a status code informing the client of the outcome of their request and also potentially how to handle it. These status codes can be broken down into 5 different ranges:

100-199 - Information Response	These are sent to tell the client the first part of their request has been accepted and they should continue sending the rest of their request. These codes are no longer very common.
200-299 - Success	This range of status codes is used to tell the client their request was successful.
300-399 - Redirection	These are used to redirect the client's request to another resource. This can be either to a different webpage or a different website altogether.
400-499 - Client Errors	Used to inform the client that there was an error with their request.
500-599 - Server Errors	This is reserved for errors happening on the server-side and usually indicate quite a major problem with the server handling the request.

♦ Common HTTP Status Codes:

There are a lot of different HTTP status codes and that's not including the fact that applications can even define their own, we'll go over the most common HTTP responses you are likely to come across:

200 - OK	The request was completed successfully.
201 - Created	A resource has been created (for example a new user or new blog post).
301 - Moved Permanently	This redirects the client's browser to a new webpage or tells search engines that the page has moved somewhere

	else and to look there instead.
302 - Found	Similar to the above permanent redirect, but as the name suggests, this is only a temporary change and it may change again in the near future.
400 - Bad Request	This tells the browser that something was either wrong or missing in their request. This could sometimes be used if the web server resource that is being requested expected a certain parameter that the client didn't send.
401 - Not Authorised	You are not currently allowed to view this resource until you have authorised with the web application, most commonly with a username and password.
403 - Forbidden	You do not have permission to view this resource whether you are logged in or not.
405 - Method Not Allowed	The resource does not allow this method request, for example, you send a GET request to the resource /create-account when it was expecting a POST request instead.
404 - Page Not Found	The page/resource you requested does not exist.
500 - Internal Service Error	The server has encountered some kind of error with your request that it doesn't know how to handle properly.
503 - Service Unavailable	This server cannot handle your request as it's either overloaded or down for maintenance.

If you are a visual learner, also check out a great http.cat (<https://http.cat/>) resource to study status codes. Now, click the "View Site" button on the right to see what some of these HTTP status messages look like in a browser.

Q1) What response code might you receive if you've created a new user or blog post article?

Answer: 201

Q2) What response code might you receive if you've tried to access a page that doesn't exist?

Answer: 404

Q3) What response code might you receive if the web server cannot access its database and the application crashes?

Answer: 503

Q4) What response code might you receive if you try to edit your profile without logging in first?

Answer: 401

>> Task 5: Headers

Headers are additional bits of data you can send to the web server when making requests.

Although no headers are strictly required when making a HTTP request, you'll find it difficult to view a website properly.

♦ Common Request Headers:

These are headers that are sent from the client (usually your browser) to the server.

Host: Some web servers host multiple websites so by providing the host headers you can tell it which one you require, otherwise you'll just receive the default website for the server.

User-Agent: This is your browser software and version number, telling the web server your browser software helps it format the website properly for your browser and also some elements of HTML, JavaScript and CSS are only available in certain browsers.

Content-Length: When sending data to a web server such as in a form, the content length tells the web server how much data to expect in the web request. This way the server can ensure it isn't missing any data.

Accept-Encoding: Tells the web server what types of compression methods the browser supports so the data can be made smaller for transmitting over the internet.

Cookie: Data sent to the server to help remember your information (see cookies task for more information).

♦ Common Response Headers:

These are the headers that are returned to the client from the server after a request.

Set-Cookie: Information to store which gets sent back to the web server on each request (see cookies task for more information).

Cache-Control: How long to store the content of the response in the browser's cache before it requests it again.

Content-Type: This tells the client what type of data is being returned, i.e., HTML, CSS, JavaScript, Images, PDF, Video, etc. Using the content-type header the browser then knows how to process the data.

Content-Encoding: What method has been used to compress the data to make it smaller when sending it over the internet.

Q1) What header tells the web server what browser is being used?

Answer: User-Agent

Q2) What header tells the browser what type of data is being returned?

Answer: Content-Type

Q3) What header tells the web server which website is being requested?

Answer: Host

>> Task 6: Cookies

You've probably heard of cookies before, they're just a small piece of data that is stored on your computer. Cookies are saved when you receive a "Set-Cookie" header from a web server. Then every further request you make, you'll send the cookie data back to the web server. Because HTTP is stateless (doesn't keep track of your previous requests), cookies can be used to remind the web server who you are, some personal settings for the website or whether you've been to the website before. Let's take a look at this as an example HTTP request:



Cookies can be used for many purposes but are most commonly used for website authentication. The cookie value won't usually be a clear-text string where you can see the password, but a token (unique secret code that isn't easily humanly guessable).

- ♦ **Viewing Your Cookies:**

You can easily view what cookies your browser is sending to a website by using the developer tools, in your browser.

Q1) Which header is used to save cookies to your computer?

Answer: Set-Cookie

>> Task 7: Making Requests

Click the "View Site" button on the right.

This is an emulator for making demo HTTP requests, using what you've learnt from the above tasks you can use it to complete the below questions.

The screenshot displays the THM Browser interface. At the top, there is a request bar with a dropdown menu set to 'GET' and a text input field containing 'http://tryhackme.com/'. To the right of the input field are a gear icon for settings and a 'Go' button. Below the request bar, the request details are shown in a dark-themed box: 'GET / HTTP/1.1', 'Host: tryhackme.com', 'User-Agent: Mozilla/5.0 Firefox/87.0', and 'Content-Length: 0'. The main area of the browser is titled 'Response' and contains the following HTTP response details: 'HTTP/1.1 200 Ok', 'Server: nginx/1.15.8', 'Thu, 2 Apr 2026 13:32:8 GMT', 'Content-Type: text/html; charset=utf-8', 'Content-Length: 228', and 'Last-Modified: Thu, 2 Apr 2026 13:32:8 GMT'. Below these details is the raw HTML response:

```
<html>
<head>
  <title>TryHackMe</title>
</head>
<body>
  Welcome to TryHackMe.com
</body>
</html>
```

 At the bottom of the browser window, there is a title bar with three colored window control buttons (red, yellow, green) and the text 'THM Browser'. The browser's address bar shows 'Welcome to TryHackMe.com'.

Q1) Make a **GET** request to **/room** page

Answer: THM{YOU'RE_IN_THE_ROOM}

URL: <http://tryhackme.com/room>

Q2) Make a **GET** request to **/blog** page and set the **id** parameter to **1**

Note: Use the gear button on the right to manage URI parameters

Answer: THM{YOU_FOUND_THE_BLOG}

URL: <http://tryhackme.com/blog?id=1>

Q3) Make a **DELETE** request to **/user/1** page

Answer: THM{USER_IS_DELETED}

URL: <http://tryhackme.com/user/1>

Q4) Make a **PUT** request to **/user/2** page with the **username** parameter set to **admin**

Note: Use the gear button on the right to manage body parameters

Answer: THM{USER_HAS_UPDATED}

Parameters: Key - username :: Value - admin

Q5) Make a **POST** request to **/login** page with the **username** of **thm** and a **password** of **letmein**

Note: Use the gear button on the right to manage body parameters

Answer: THM{HTTP_REQUEST_MASTER}

Parameters: Key - username :: Value - thm

Parameters: Key - password :: Value - letmein